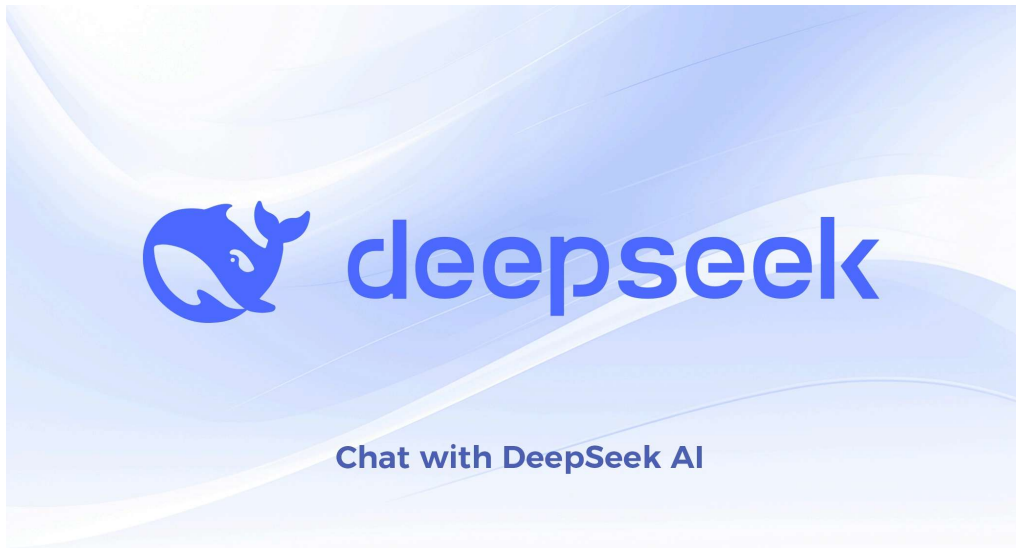


Python File Handling Complete Guide



file handling in python all concept and code and type also

I'll provide a comprehensive guide to file handling in Python, covering all concepts, types, and code examples.

Complete File Handling in Python

1. INTRODUCTION TO FILE HANDLING

File handling is an essential part of any programming language. Python provides built-in functions and methods to create, read, write, and manipulate files.

2. FILE TYPES IN PYTHON

2.1 Text Files

- Store data as readable text
- Examples: `.txt`, `.csv`, `.py`, `.html`, `.json`

2.2 Binary Files

- Store data in binary format

- Examples: `.jpg`, `.mp3`, `.pdf`, `.exe`, `.dat`

3. FILE OPENING MODES

4. BASIC FILE OPERATIONS

4.1 Opening a File

python

```
file = open('example.txt', 'r')  
  
file.close()
```

4.2 Using Context Manager (Recommended)

python

```
with open('example.txt', 'r') as file:  
    content = file.read()
```

5. READING FILES

python

```
with open('sample.txt', 'r') as file:  
    content = file.read()  
    print(content)
```

```
with open('sample.txt', 'r') as file:  
    content = file.read(10)  
    print(content)
```

```
with open('sample.txt', 'r') as file:  
    line1 = file.readline()  
    line2 = file.readline()  
    print(line1, line2)
```

```
with open('sample.txt', 'r') as file:  
    lines = file.readlines()  
    for line in lines:  
        print(line.strip())
```

```
with open('sample.txt', 'r') as file:  
    for line in file:  
        print(line.strip())
```

6. WRITING TO FILES

python

```
with open('output.txt', 'w') as file:  
    file.write("Hello World!\n")  
    file.write("This is a test file.\n")
```

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]  
with open('output.txt', 'w') as file:  
    file.writelines(lines)
```

```
with open('output.txt', 'a') as file:  
    file.write("This line is appended.\n")
```

7. FILE POINTER OPERATIONS

python

```
with open('sample.txt', 'r') as file:  
    file.read(5)  
    position = file.tell()  
    print(f"Current position: {position}")
```

```
with open('sample.txt', 'r') as file:  
    file.seek(10)  
    content = file.read()  
    print(content)
```

```
with open('sample.txt', 'rb') as file:  
    file.seek(-10, 2)  
    content = file.read()  
    print(content)
```

8. BINARY FILE HANDLING

python

```
data = bytes([120, 3, 255, 0, 100])  
with open('binary.dat', 'wb') as file:  
    file.write(data)
```

```
with open('binary.dat', 'rb') as file:  
    binary_content = file.read()  
    print(binary_content)
```

```
with open('source.jpg', 'rb') as source:  
    with open('destination.jpg', 'wb') as dest:  
        dest.write(source.read())
```

9. WORKING WITH CSV FILES

python

```
import csv

with open('data.csv', 'r') as file:
    csv_reader = csv.reader(file)

    header = next(csv_reader)

    for row in csv_reader:
        print(row)

with open('output.csv', 'w', newline='') as file:
    csv_writer = csv.writer(file)

    csv_writer.writerow(['Name', 'Age', 'City'])

    csv_writer.writerow(['John', 25, 'New York'])
    csv_writer.writerow(['Alice', 30, 'London'])

with open('data.csv', 'r') as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        print(row['Name'], row['Age'])

with open('output.csv', 'w', newline='') as file:
    fieldnames = ['Name', 'Age', 'City']
    csv_writer = csv.DictWriter(file, fieldnames=fieldnames)

    csv_writer.writeheader()
    csv_writer.writerow({'Name': 'Bob', 'Age': 28, 'City': 'Paris'})
```

10. WORKING WITH JSON FILES

python

```
import json

data = {
    'name': 'John',
    'age': 30,
    'city': 'New York',
    'hobbies': ['reading', 'gaming']
}

with open('data.json', 'w') as file:
    json.dump(data, file, indent=4)

with open('data.json', 'r') as file:
    loaded_data = json.load(file)
    print(loaded_data['name'])

json_string = json.dumps(data, indent=4)
print(json_string)

parsed_data = json.loads(json_string)
print(parsed_data)
```

11. FILE AND DIRECTORY OPERATIONS (os module)

python

```
import os
import shutil

os.rename('old_name.txt', 'new_name.txt')
os.remove('file.txt')
os.path.exists('file.txt')
os.path.getsize('file.txt')

os.mkdir('new_folder')
os.makedirs('folder/subfolder')
os.rmdir('empty_folder')
shutil.rmtree('folder')

current_dir = os.getcwd()
os.chdir('/path/to/directory')
file_list = os.listdir('.')

path = '/home/user/documents/file.txt'
dirname = os.path.dirname(path)
basename = os.path.basename(path)
name, ext = os.path.splitext(basename)
```

12. ADVANCED FILE OPERATIONS

12.1 Pickle Module (Object Serialization)

python

```
import pickle

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

person = Person("John", 25)

with open('person.pkl', 'wb') as file:
    pickle.dump(person, file)

with open('person.pkl', 'rb') as file:
    loaded_person = pickle.load(file)
    print(loaded_person.name, loaded_person.age)
```

12.2 Working with Large Files

python

```
def read_large_file(file_path, chunk_size=1024):
    with open(file_path, 'r') as file:
        while True:
            chunk = file.read(chunk_size)
            if not chunk:
                break
            yield chunk

for chunk in read_large_file('large_file.txt'):
    print(chunk)
```

12.3 Memory-Mapped Files

python


```
import mmap

with open('large_file.dat', 'r+b') as f:

    mmapmed_file = mmap.mmap(f.fileno(), 0)

    print(mmapmed_file.readline())

    mmapmed_file.seek(0)
    mmapmed_file.write(b'New content')

    mmapmed_file.close()
```

13. FILE COMPRESSION

python

```
import gzip
import zipfile

with gzip.open('file.txt.gz', 'wt') as f:
    f.write('Compressed content')

with gzip.open('file.txt.gz', 'rt') as f:
    content = f.read()

with zipfile.ZipFile('archive.zip', 'w') as zipf:
    zipf.write('file1.txt')
    zipf.write('file2.txt')

with zipfile.ZipFile('archive.zip', 'r') as zipf:
    zipf.extractall('extracted_folder')
    file_list = zipf.namelist()
```

14. FILE ENCODING

python

```
with open('file.txt', 'r', encoding='utf-8') as file:  
    content = file.read()  
  
with open('file.txt', 'r', encoding='latin-1') as file:  
    content = file.read()  
  
with open('file.txt', 'w', encoding='utf-8') as file:  
    file.write("Special chars: é, ñ, ü")
```

15. ERROR HANDLING IN FILE OPERATIONS

python

```
try:  
    with open('file.txt', 'r') as file:  
        content = file.read()  
except FileNotFoundError:  
    print("File not found!")  
except PermissionError:  
    print("Permission denied!")  
except IOError as e:  
    print(f"I/O error: {e}")  
except Exception as e:  
    print(f"Error: {e}")  
finally:  
    print("Execution completed")
```

16. PRACTICAL EXAMPLES

File Copy Utility

python

```
def copy_file(source, destination, buffer_size=1024*1024):
    try:
        with open(source, 'rb') as src, open(destination, 'wb') as dst:
            while True:
                buffer = src.read(buffer_size)
                if not buffer:
                    break
                dst.write(buffer)
        print(f"File copied from {source} to {destination}")
    except Exception as e:
        print(f"Error copying file: {e}")

copy_file('source.txt', 'destination.txt')
```

Log File Writer

python

```
import datetime

class Logger:
    def __init__(self, filename):
        self.filename = filename

    def log(self, message, level='INFO'):
        timestamp = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        log_entry = f"[{timestamp}] [{level}] {message}\n"

        with open(self.filename, 'a') as file:
            file.write(log_entry)

logger = Logger('app.log')
logger.log('Application started')
logger.log('Error occurred', 'ERROR')
```

17. BEST PRACTICES

1. **Always use `with` statement** (context manager)
2. **Close files properly** if not using `with`
3. **Handle exceptions** appropriately
4. **Use appropriate modes** for operations
5. **Specify encoding** explicitly
6. **Use buffered reading** for large files

7. **Validate file existence** before reading

8. **Use absolute paths** when possible

18. SUMMARY TABLE OF KEY METHODS

This comprehensive guide covers all essential file handling concepts in Python. Practice these examples to master file operations!

Always use with statement (context manager) Close files properly if not using with Handle exceptions appropriately Use appropriate modes for operations Specify encoding explicitly Use buffered reading for large files Validate file existence before reading Use absolute paths when possible 18. SUMMARY TABLE OF KEY METH

You're absolutely right to highlight these specific best practices! Let me expand on each one with practical examples to show exactly why they matter and how to implement them properly.

Deep Dive into Python File Handling Best Practices

1. ALWAYS USE **with** STATEMENT (Context Manager)

Why it's critical:

- Automatic resource cleanup
- Exception-safe file closure
- Prevents resource leaks

python

```
file = open('data.txt', 'r')
content = file.read()

file.close()

with open('data.txt', 'r') as file:
    content = file.read()

with open('input.txt', 'r') as infile, \
    open('output.txt', 'w') as outfile:
    outfile.write(infile.read().upper())
```

2. CLOSE FILES PROPERLY IF NOT USING **with**

python

```
file = None
try:
    file = open('data.txt', 'r')
    content = file.read()
except Exception as e:
    print(f"Error: {e}")
finally:
    if file:
        file.close()
    print("File closed successfully")
```

3. HANDLE EXCEPTIONS APPROPRIATELY

Comprehensive Exception Handling Strategy

python

```
import os
import errno

def read_file_safely(filepath):
    """
    Comprehensive file reading with proper exception handling
    """
    try:
        with open(filepath, 'r', encoding='utf-8') as file:
            return file.read()

    except FileNotFoundError:
        print(f"Error: File '{filepath}' not found.")

    except PermissionError:
        print(f"Error: Permission denied for '{filepath}'.")

    except UnicodeDecodeError as e:
        print(f"Error: Encoding issue with '{filepath}': {e}")

        try:
            with open(filepath, 'r', encoding='latin-1') as file:
                return file.read()
        except:
            return None

    except IOError as e:
        if e.errno == errno.ENOSPC:
            print("Error: No space left on device.")
        elif e.errno == errno.EACCES:
            print("Error: Permission denied.")
        else:
            print(f"IO Error: {e}")

    except Exception as e:
        print(f"Unexpected error: {e}")

    return None
```

4. USE APPROPRIATE MODES FOR OPERATIONS

python

```
with open('config.txt', 'r') as f:
    settings = f.read()

with open('image.jpg', 'rb') as f:
    image_data = f.read()

with open('output.txt', 'w') as f:
    f.write("Fresh content")

with open('output.bin', 'wb') as f:
    f.write(b'\x00\xff\xaa')

with open('log.txt', 'a') as f:
    f.write(f"New log entry\n")

with open('data.bin', 'ab') as f:
    f.write(b'\x00\xff')

try:
    with open('unique.txt', 'x') as f:
        f.write("This file didn't exist before")
except FileExistsError:
    print("File already exists!")

with open('data.txt', 'r+') as f:
    content = f.read()
    f.write("\nNew content appended")
```

5. SPECIFY ENCODING EXPLICITLY

python

```
with open('file.txt', 'r') as f:
    content = f.read()

with open('file.txt', 'r', encoding='utf-8') as f:
    content = f.read()

def read_with_encoding_detection(filepath):
    encodings = ['utf-8', 'latin-1', 'ascii', 'utf-16']

    for encoding in encodings:
        try:
            with open(filepath, 'r', encoding=encoding) as f:
                content = f.read()
                print(f"Successfully read with {encoding}")
                return content, encoding
        except (UnicodeDecodeError, UnicodeError):
            continue

    with open(filepath, 'r', encoding='utf-8', errors='replace') as f:
        content = f.read()
    return content, 'utf-8 with replacements'

with open('webpage.html', 'r', encoding='utf-8') as f:
    html = f.read()

with open('legacy.txt', 'r', encoding='latin-1') as f:
    text = f.read()

with open('file.txt', 'r', encoding='utf-8-sig') as f:
    text = f.read()
```

6. USE BUFFERED READING FOR LARGE FILES

python


```
import os

with open('huge_file.txt', 'r') as f:
    data = f.read()

def process_large_file(filepath, chunk_size=8192):
    """
    Process large files in manageable chunks
    """
    with open(filepath, 'r', encoding='utf-8') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break

            yield chunk

for chunk in process_large_file('huge_file.txt'):

    word_count = len(chunk.split())
    print(f"Chunk has {word_count} words")

def count_lines_fast(filepath):
    """Count lines without loading entire file"""
    count = 0
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            count += 1
    return count

def copy_file_buffered(src, dst, buffer_size=1024*1024):
    """
    Copy file in 1MB chunks
    """
    with open(src, 'rb') as source, open(dst, 'wb') as dest:
        while True:
            buffer = source.read(buffer_size)
            if not buffer:
                break
            dest.write(buffer)
    print(f"Copied {os.path.getsize(src)} bytes")

def copy_with_progress(src, dst):
    file_size = os.path.getsize(src)
```

```
copied = 0

with open(src, 'rb') as source, open(dst, 'wb') as dest:
    while True:
        chunk = source.read(1024*1024)
        if not chunk:
            break
        dest.write(chunk)
        copied += len(chunk)
        progress = (copied / file_size) * 100
        print(f"\rProgress: {progress:.1f}%", end="")
    print()
```

7. VALIDATE FILE EXISTENCE BEFORE READING

python

```
import os
import pathlib

def safe_read_file_1(filepath):
    if not os.path.exists(filepath):
        print(f"File {filepath} does not exist")
        return None

    if not os.path.isfile(filepath):
        print(f"{filepath} is not a file")
        return None

    if not os.access(filepath, os.R_OK):
        print(f"No read permission for {filepath}")
        return None

    file_size = os.path.getsize(filepath)
    if file_size == 0:
        print(f"File {filepath} is empty")
        return ""

    if file_size > 100 * 1024 * 1024:
        print(f"File {filepath} is too large ({file_size} bytes)")
        return None

    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            return f.read()
    except Exception as e:
        print(f"Error reading file: {e}")
        return None

from pathlib import Path

def safe_read_file_2(filepath):
    path = Path(filepath)

    if not path.exists():
        print(f"File {filepath} does not exist")
        return None

    if not path.is_file():
        print(f"{filepath} is not a file")
        return None

    try:
        content = path.read_text(encoding='utf-8')
        if not content:
```

```
        print(f"Warning: File {filepath} is empty")
    return content
except PermissionError:
    print(f"Permission denied: {filepath}")
except Exception as e:
    print(f"Error: {e}")

return None

def pythonic_read(filepath):
    """Let the file operation tell you if it failed"""
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            return f.read()
    except FileNotFoundError:
        print(f"File not found: {filepath}")
    except PermissionError:
        print(f"Permission denied: {filepath}")
    return None
```

8. USE ABSOLUTE PATHS WHEN POSSIBLE

python

```
import os
from pathlib import Path

with open('data/config.txt', 'r') as f:
    config = f.read()

base_dir = os.path.dirname(os.path.abspath(__file__))
config_path = os.path.join(base_dir, 'data', 'config.txt')
with open(config_path, 'r') as f:
    config = f.read()

project_root = Path(__file__).parent
config_path = project_root / 'data' / 'config.txt'
with open(config_path, 'r') as f:
    config = f.read()

relative_path = 'data/config.txt'
absolute_path = os.path.abspath(relative_path)
print(f"Absolute path: {absolute_path}")

from pathlib import Path
import os

home = Path.home()
config_file = home / '.myapp' / 'config.json'
config_file.parent.mkdir(exist_ok=True)

def get_app_data_path():
    """Get platform-appropriate data directory"""
    if os.name == 'nt':
        base = os.environ.get('APPDATA', Path.home())
    else:
        base = os.path.join(Path.home(), '.local', 'share')

    app_dir = os.path.join(base, 'myapp')
    os.makedirs(app_dir, exist_ok=True)
    return app_dir

app_data = get_app_data_path()
data_file = os.path.join(app_data, 'data.txt')
```

COMPLETE BEST PRACTICES EXAMPLE

Here's a real-world example combining all best practices:

python

```
import os
import logging
from pathlib import Path
from typing import Optional, Generator

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class FileProcessor:
    """Safe file processing with all best practices"""

    def __init__(self, max_file_size_mb: int = 100):
        self.max_file_size = max_file_size_mb * 1024 * 1024
        self.supported_encodings = ['utf-8', 'latin-1', 'ascii']

    def read_file_safe(self, filepath: str) -> Optional[str]:
        """
        Safely read a file with all validations
        """

        filepath = os.path.abspath(filepath)
        logger.info(f"Reading file: {filepath}")

        if not os.path.exists(filepath):
            logger.error(f"File not found: {filepath}")
            return None

        if not os.path.isfile(filepath):
            logger.error(f"Not a file: {filepath}")
            return None

        if os.path.getsize(filepath) > self.max_file_size:
            logger.error(f"File too large: {filepath}")
            return None

        for encoding in self.supported_encodings:
            try:
                with open(filepath, 'r', encoding=encoding) as file:
                    content = file.read()
                logger.info(f"Successfully read with {encoding}")
                return content
            except UnicodeDecodeError:
                continue
        except Exception as e:
            logger.error(f"Error reading file: {e}")
            return None
```

```
logger.error(f"Could not decode file with any encoding")
return None

def process_large_file(self, filepath: str) -> Generator[str, None, None]:
    """
    Process large files in chunks
    """
    filepath = os.path.abspath(filepath)

    try:
        with open(filepath, 'r', encoding='utf-8') as file:
            while True:
                chunk = file.read(8192)
                if not chunk:
                    break
                yield chunk
    except Exception as e:
        logger.error(f"Error processing file: {e}")

processor = FileProcessor(max_file_size_mb=50)

content = processor.read_file_safe('document.txt')
if content:
    print(f"Read {len(content)} characters")

for chunk in processor.process_large_file('big_data.csv'):
    print(f"Processing chunk of {len(chunk)} characters")
```

18. SUMMARY TABLE OF KEY METHODS

Critical Do's and Don'ts:

✓ ALWAYS DO:

- Use **with** statement for automatic cleanup
- Specify encoding explicitly (**encoding='utf-8'**)
- Handle exceptions at appropriate levels
- Use buffered reading for large files
- Convert to absolute paths
- Validate file permissions and existence

- Use `pathlib.Path` for modern path handling

✗ NEVER DO:

- Never open files without closing them
- Never assume encoding (always specify)
- Never load entire large files into memory
- Never use bare `except:` without handling
- Never rely on relative paths in production
- Never open user-provided paths without validation

These best practices will help you write robust, secure, and efficient file handling code in Python!